

Visión Artificial

Tema 4. Apéndice

Algoritmos de estimación robusta: Transformada de Hough (HT) y *Random Sample Consensus* (RANSAC)

PEDRO ENRIQUE LÓPEZ DE TERUEL ALCOLEA

DEPARTAMENTO INGENIERÍA Y TECNOLOGÍA DE COMPUTADORES
FACULTAD DE INFORMÁTICA

Universidad de Murcia

Máster en Inteligencia Artificial

2025/2026



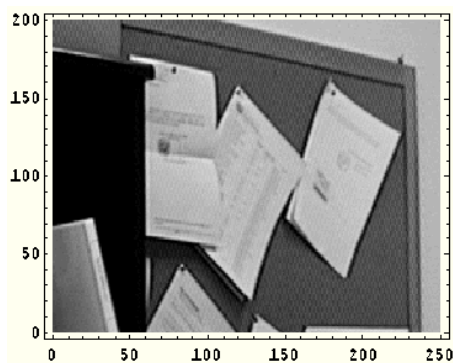
- ▶ Estimación robusta: estimación de modelos de explicación de unos datos a partir de **datos ruidosos** (con posibles *outliers*).
- ▶ Dos procedimientos muy populares en la comunidad de visión:
 - **RANSAC** (*Random Sample Consensus*): técnica de estimación estadística robusta genérica.
 - La **HT** (*Hough transform*): detector de modelos más o menos complejos en un determinado espacio de entrada.
 - RANSAC y HT usados inicialmente para detectar rectas en imágenes...
 - ...pero en realidad son técnicas de estimación estadística robusta generales extrapolables a otros muchos problemas (de visión artificial, pero también en otros ámbitos).

Apéndice T4: Estimación robusta

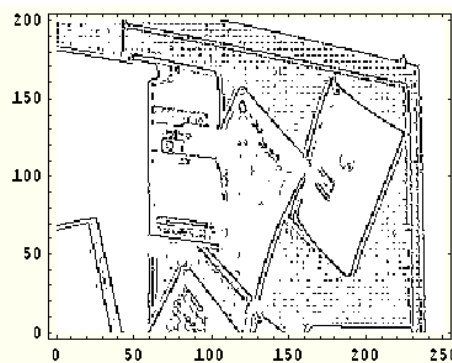
Búsqueda robusta en espacios paramétricos



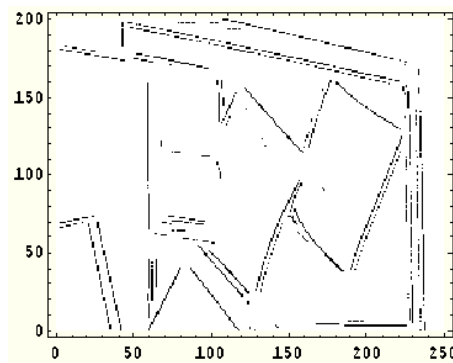
- Lo que RANSAC y HT comparten:
 - Ajuste de *features* como búsqueda en espacio paramétrico:
 - Escogemos un modelo parametrizado para representar un modelo, y buscamos instancia(s) de ello(s) en unos datos de entrada
 - **Ejemplo:** búsqueda de segmentos en imágenes binarias de borde, con posibles *outliers*.



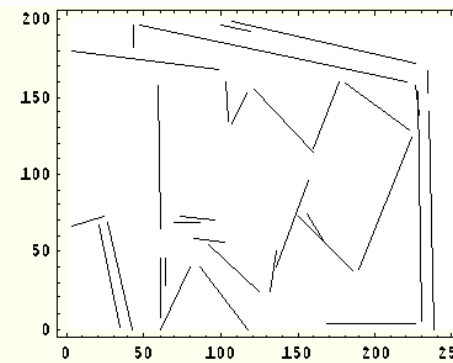
Original



Puntos de borde (Canny)



Inliers



Segmentos



- ▶ El criterio de búsqueda **NO** es estrictamente local:
 - No se sabe si un punto pertenece o no a un modelo sin mirar más puntos.
- ▶ Hay que resolver dos cuestiones simultáneamente:
 - **Asociación**: ¿Cuántas instancias del modelo hay en los puntos de entrada, y qué instancias del modelo se corresponden con cada punto de entrada?
 - **Estimación**: ¿Qué modelo (o conjunto de modelos) representa mejor a un conjunto de puntos de entrada?

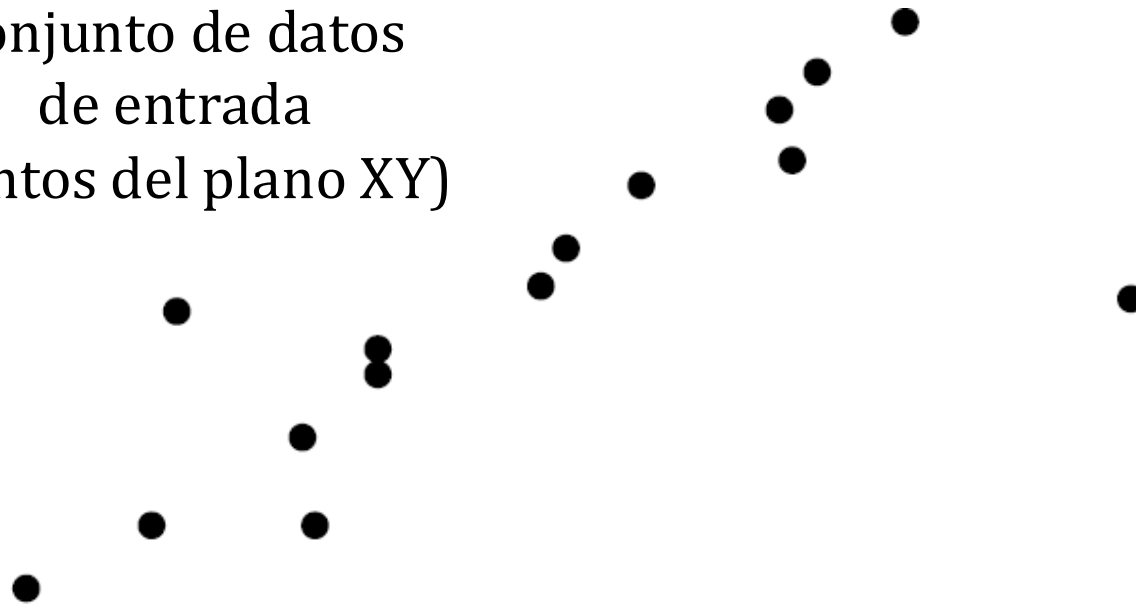
Apéndice T4: Estimación robusta

Algoritmo básico RANSAC



- Ejemplo de búsqueda de recta (modelo) que explica un conjunto de puntos 2D de entrada (datos):

Conjunto de datos
de entrada
(puntos del plano XY)

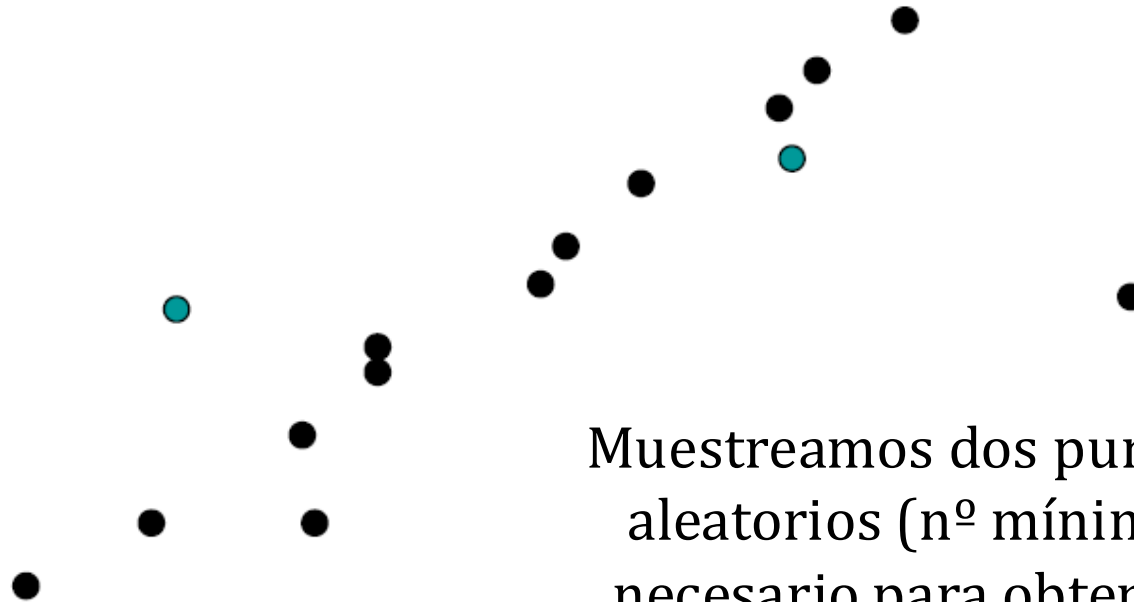


Apéndice T4: Estimación robusta

Algoritmo básico RANSAC



- Ejemplo de búsqueda de recta (modelo) que explica un conjunto de puntos 2D de entrada (datos):



Muestreamos dos puntos
aleatorios (n° mínimo
necesario para obtener
una recta única)

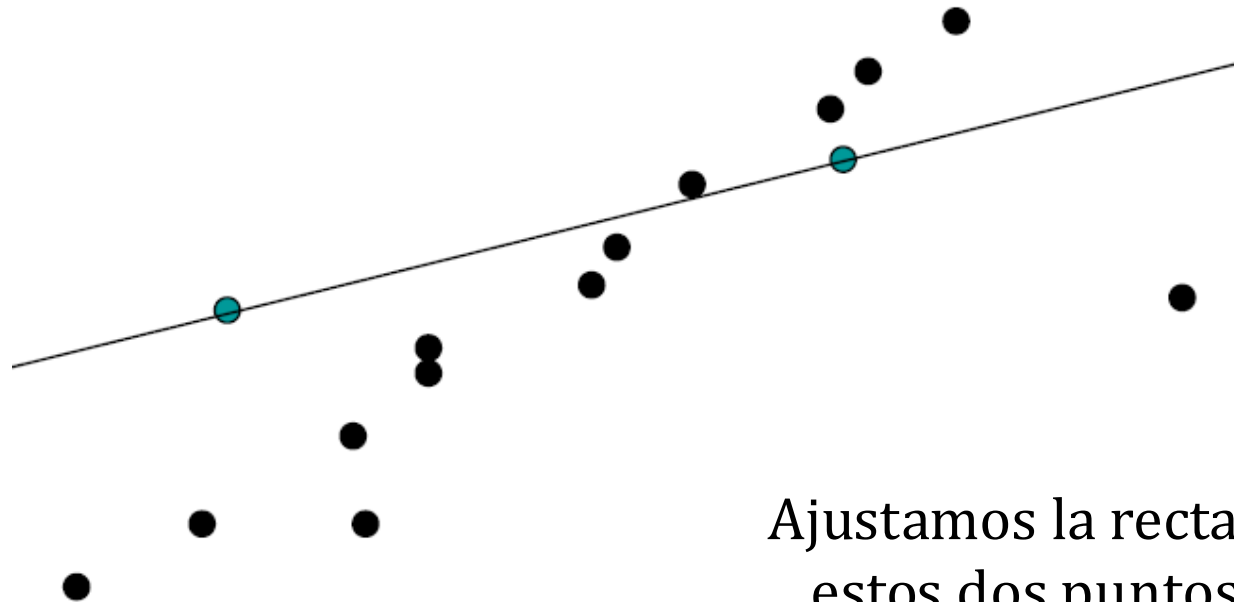
Apéndice T4: Estimación robusta

Algoritmo básico RANSAC



Máster en
Inteligencia
Artificial
UMU

- Ejemplo de búsqueda de recta (modelo) que explica un conjunto de puntos 2D de entrada (datos):



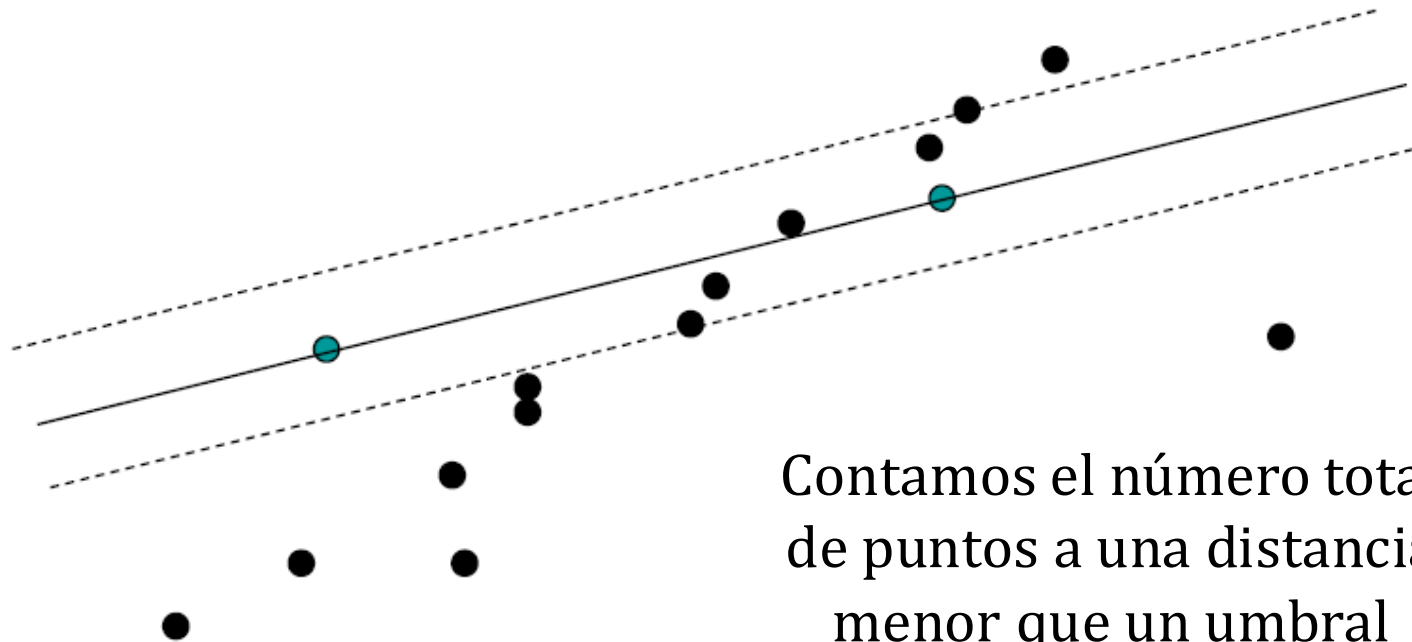
Ajustamos la recta a
estos dos puntos

Apéndice T4: Estimación robusta

Algoritmo básico RANSAC



- Ejemplo de búsqueda de recta (modelo) que explica un conjunto de puntos 2D de entrada (datos):



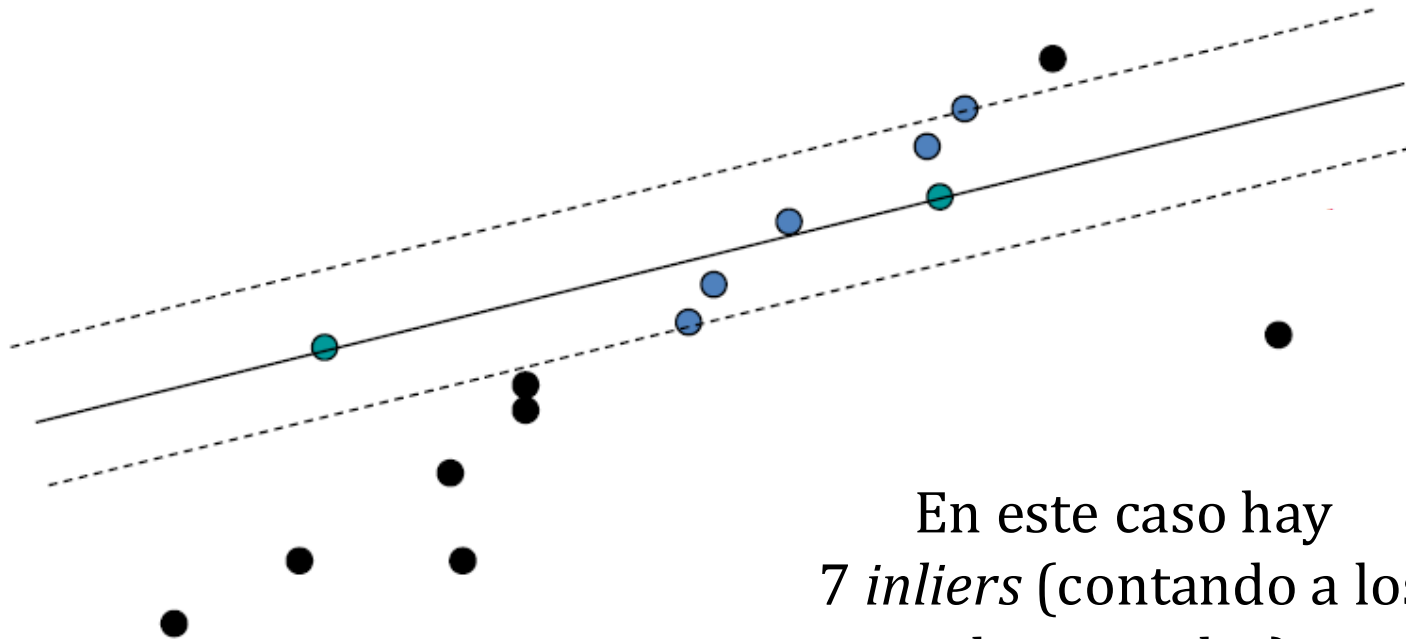
Contamos el número total de puntos a una distancia menor que un umbral de la recta estimada

Apéndice T4: Estimación robusta

Algoritmo básico RANSAC



- Ejemplo de búsqueda de recta (modelo) que explica un conjunto de puntos 2D de entrada (datos):



En este caso hay
7 *inliers* (contando a los
dos iniciales)

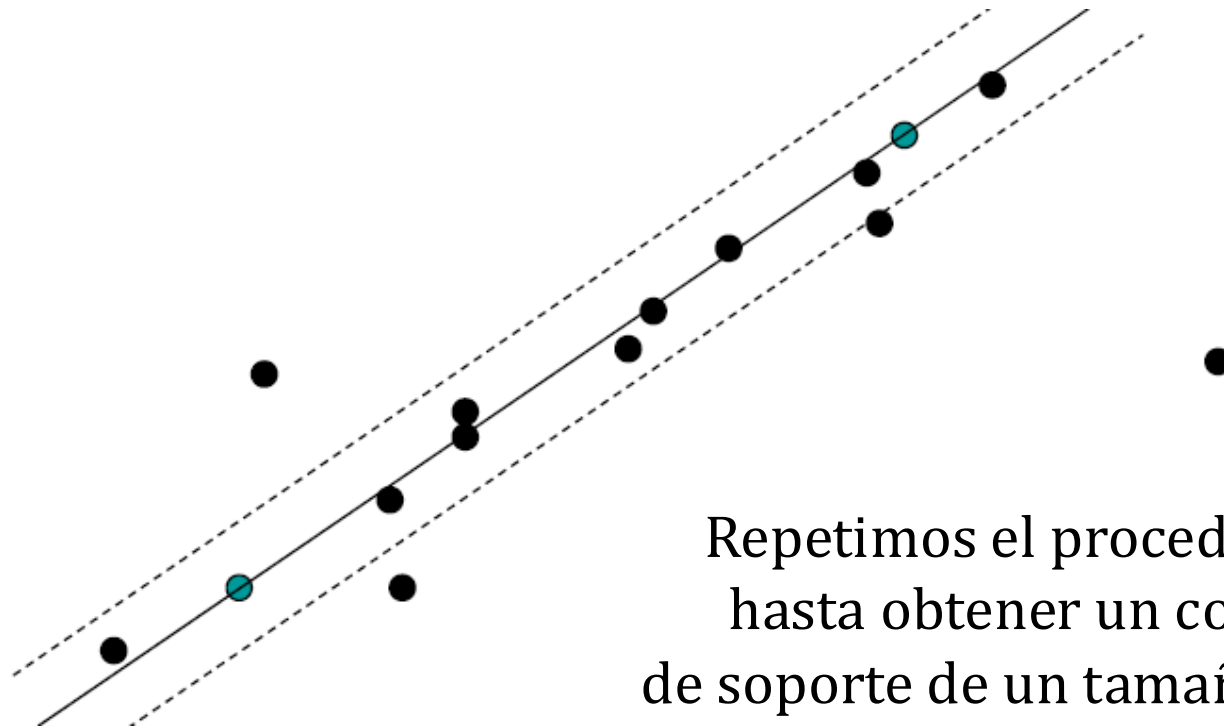
Apéndice T4: Estimación robusta

Algoritmo básico RANSAC



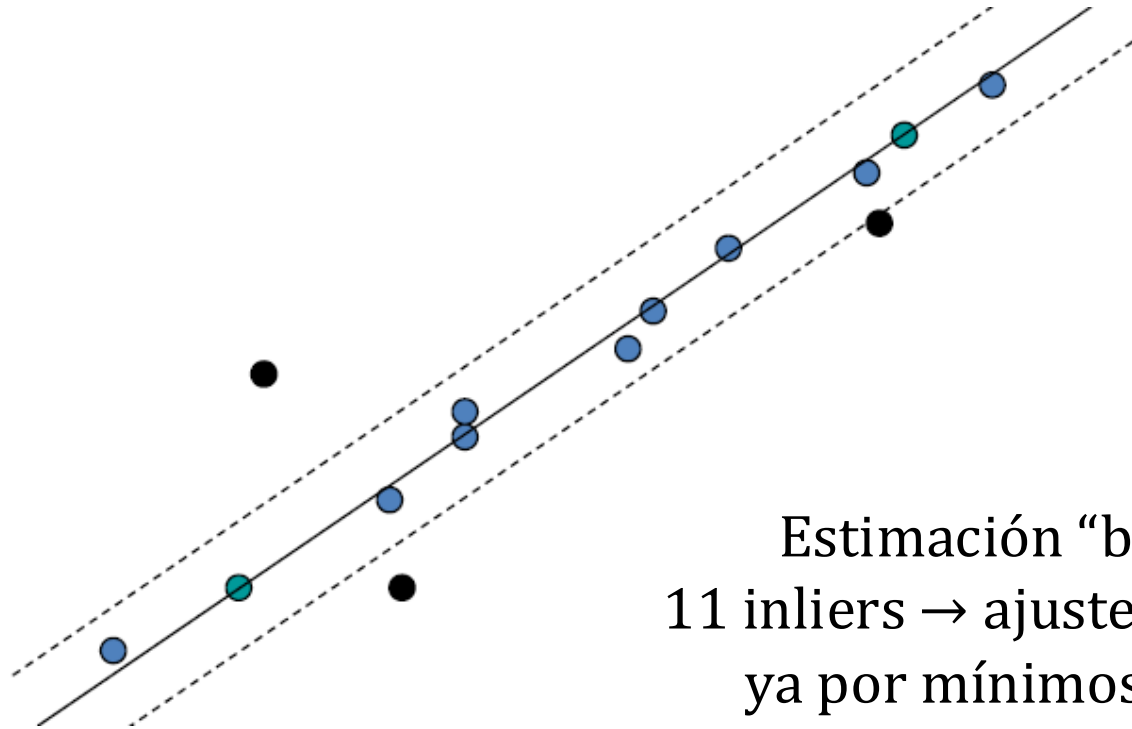
Máster en
Inteligencia
Artificial
UMU

- Ejemplo de búsqueda de recta (modelo) que explica un conjunto de puntos 2D de entrada (datos):



Repetimos el procedimiento
hasta obtener un conjunto
de soporte de un tamaño mínimo
deseado

- Ejemplo de búsqueda de recta (modelo) que explica un conjunto de puntos 2D de entrada (datos):

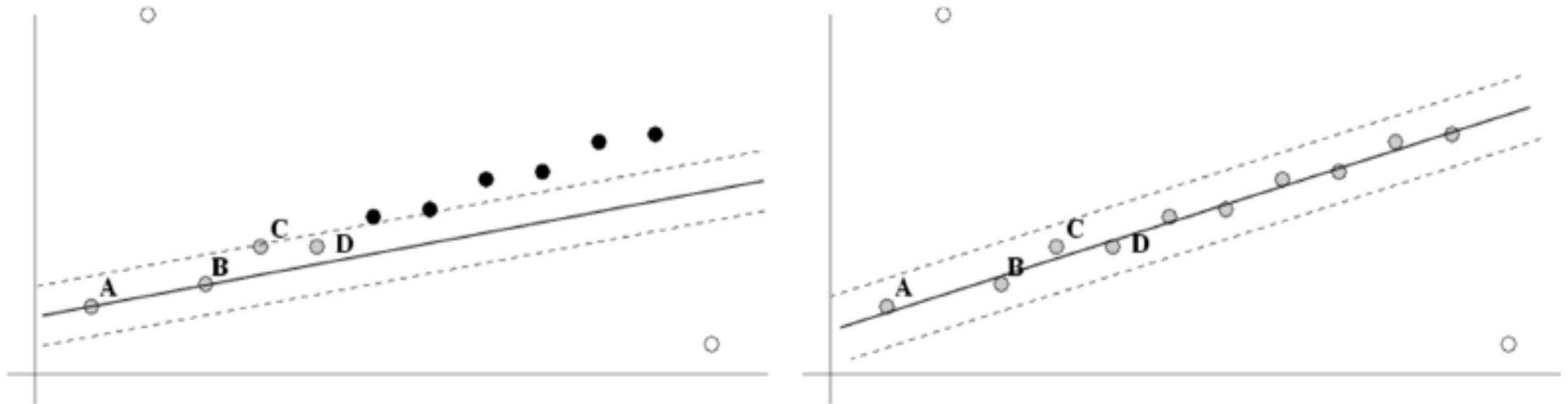


Apéndice T4: Estimación robusta

Iteración hasta convergencia



- Incluso después de haber localizado un conjunto de *inliers*, la cantidad de éstos puede variar tras la reestimación por mínimos cuadrados → iterar hasta convergencia:





- ▶ **RANSAC:** “consenso en muestreo aleatorio”
- ▶ Queremos evitar el impacto de los *outliers* → busquemos *inliers* para cada recta.
- ▶ Intuición:
 - Si cogemos algún *outlier* y estimamos una recta usándolo, la recta conseguida tendrá “poco soporte” en los datos.

Apéndice T4: Estimación robusta

Algoritmo básico RANSAC



Paso 1: Seleccionar aleatoriamente un conjunto de puntos semilla, con el tamaño mínimo necesario para realizar una estimación del modelo (2 puntos (x,y) para el problema de estimar rectas del plano).

Paso 2: Estimar el modelo (recta 2D) a partir del conjunto anterior.

Paso 3: Buscar *inliers* del modelo estimado (conjunto de “puntos de soporte” bien explicados por la recta encontrada).

Paso 4: Si hay un número suficiente de *inliers*, recomputar el modelo (la recta, por mínimos cuadrados) usando sólo los *inliers*. En otro caso, volver al paso 1.

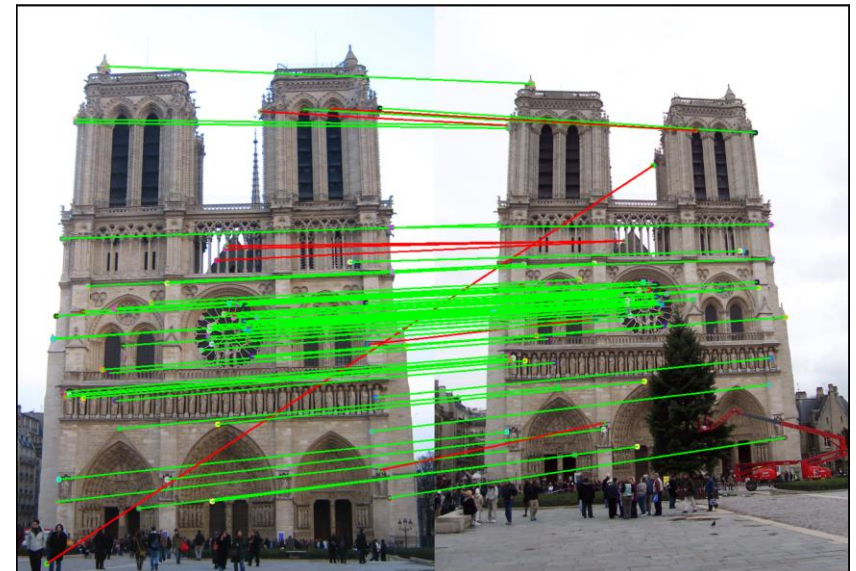
- (Este algoritmo sólo encuentra un segmento; para localizar más, se borrarían los *inliers* de la recta encontrada, y se volvería a empezar).

Apéndice T4: Estimación robusta

Genericidad de RANSAC



- ▶ El algoritmo anterior estima un modelo plausible para explicar unos datos con posibles *outliers*, **independientemente del modelo y los datos concretos**.
- ▶ P.e., para el caso de la búsqueda de segmentos en una imagen, cada modelo es un segmento, y los datos son simples puntos 2D...
- ▶ ... pero, p.e., los datos podrían ser *matchings* entre puntos de dos imágenes distintas, e.d., 4-tuplas de la forma $(x1,y1,x2,y2)$, y el modelo a buscar una transformación homográfica que lleva de una a otra imagen.
 - En ese caso ¿cuál sería el mínimo número de *matchings* necesarios? →
 - 4 *matchings* para una H general, sólo 3 si limitamos la H a que sea afín (sin perspectiva), 2 únicamente si la limitamos a una similaridad (traslación, rotación, escala).
 - Lo usaremos más adelante, al ver reconocimiento de objetos con SIFT.
- ▶ Por tanto, RANSAC es un potente método genérico de estimación robusta (en el sentido de que puede trabajar con *outliers*).



Inliers / *Outliers*

► ¿Cuántos muestreos independientes se necesitan?

- Sea w [0.0-1.0] la proporción de inliers.
- Sea n el mínimo número de datos para calcular un modelo (2 en el caso de las rectas a partir de puntos).
- Sea k el número de muestreos realizados.

► Entonces:

- La probabilidad de que un muestreo completo sea correcto es w^n .
- La probabilidad de que fallen todos los muestreos es $(1-w^n)^k$ → hay que coger k suficientemente grande para que dicho valor esté por debajo de un umbral deseado (probabilidad de fallo).

Apéndice T4: Estimación robusta

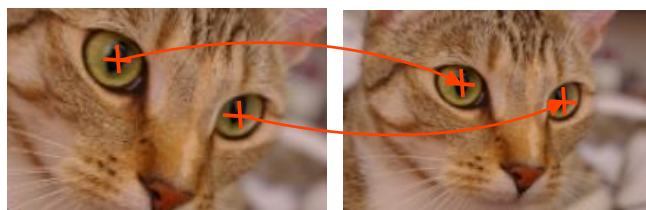
Número de muestreos necesarios



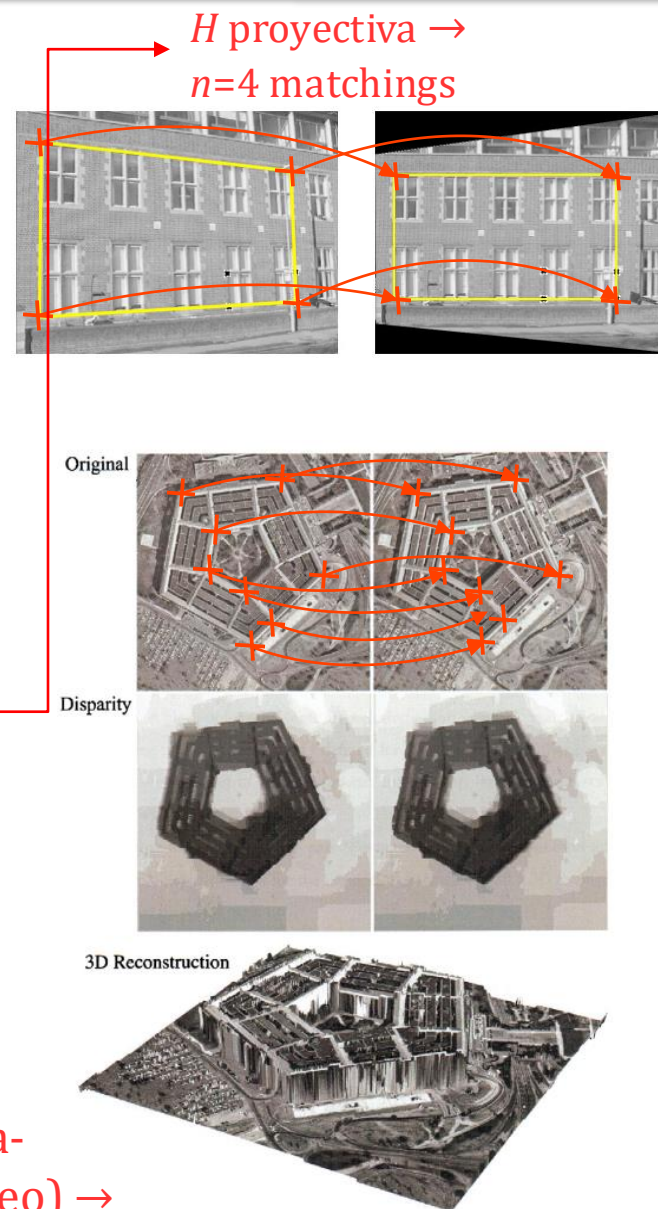
- Ejemplo muestreos mínimos (k) para probabilidad de fallo < 0.01 , para distintos valores de n y w :

Tamaño muestra (n)	Proporción outliers ($1-w$)						
	5%	10%	20%	25%	30%	40%	50%
2	2	3	5	6	7	11	17
3	3	4	7	9	11	19	35
4	3	5	9	13	17	34	72
5	4	6	12	17	26	57	146
6	4	7	16	24	37	97	293
7	4	8	20	33	54	163	588
8	5	9	26	44	78	272	1177

Rectas /
 H similaridad $\rightarrow$
 $n=2$ matchings:



matriz funda-
mental (stereo) $\rightarrow$
 $n=8$ matchings:



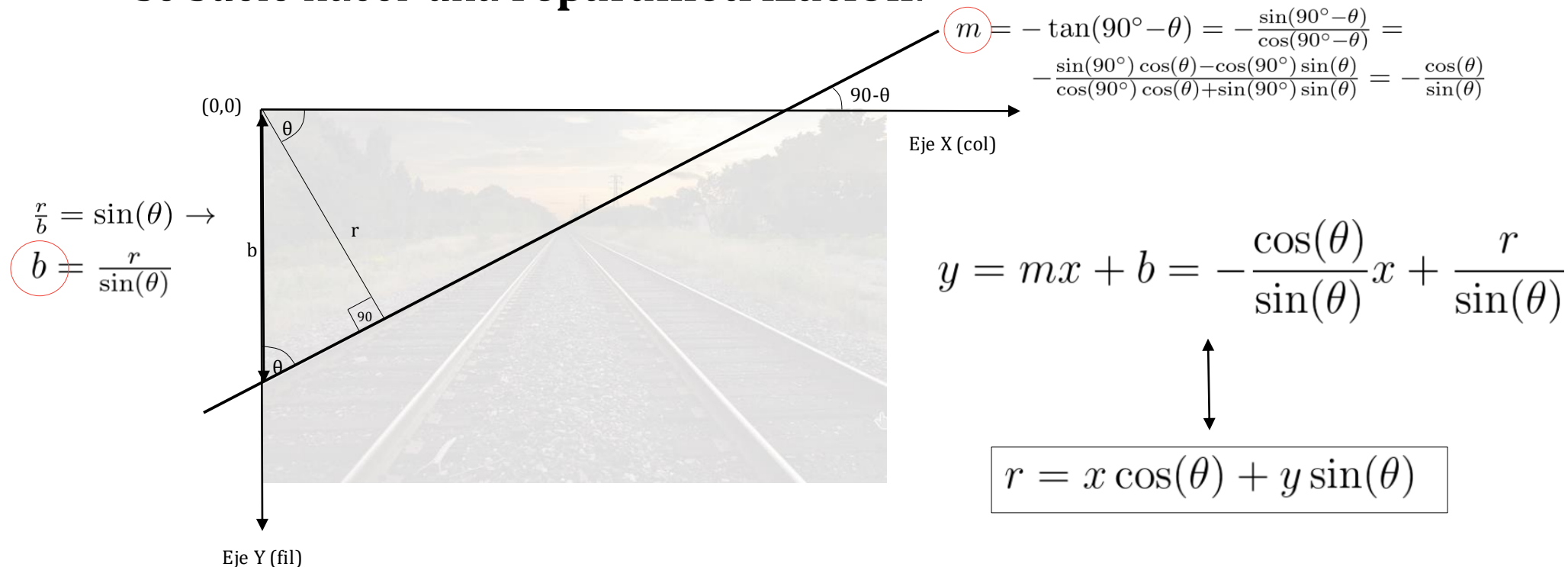
- ▶ Técnica estadística clásica:
 - Hough (1964), Duda & Hart (1972), Ballard (1981).
- ▶ El propósito de esta técnica es encontrar instancias imperfectas de objetos de una determinada forma parametrizable, mediante un **esquema de votación**.
- ▶ Idea principal:
 - Registrar todas las posibles rectas (modelos) en las que podría estar cada punto.
 - Buscar rectas (modelos) que obtienen muchos votos.
- ▶ La votación se lleva a cabo en el **espacio de parámetros** (=acumulador), y las formas candidatas serán máximos de dicho espacio, convenientemente discretizado.

Apéndice T4: Estimación robusta

Detección de rectas con HT



- Caso más simple: detección de rectas $y=mx+b$:
- Para evitar problemas con rectas casi verticales (m muy grande), se suele hacer una **reparametrización**:



El parámetro r representa la distancia de la línea al origen, y θ el ángulo que forma con el eje X la normal a la recta trazada desde el origen.

- Un número infinito de líneas pueden pasar por cada punto del plano (*pencil of lines*). Si tenemos un punto (x_0, y_0) de la imagen, todas las líneas que pasan por él satisfacen la ecuación:

$$r(\theta) = x_0 \cos(\theta) + y_0 \sin(\theta)$$

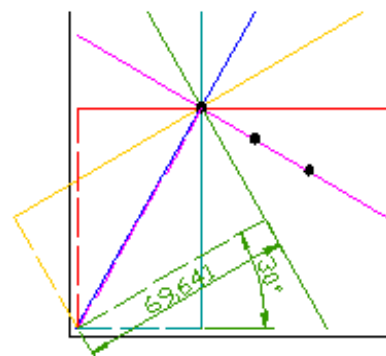
- Esto se corresponde con una *curva sinusoidal* en el plano (r, θ) , única para cada punto.
- Este plano (r, θ) , será nuestro espacio paramétrico (donde se realizará la “votación”).

Apéndice T4: Estimación robusta

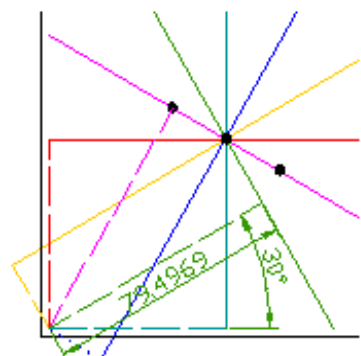
Detección de rectas con HT



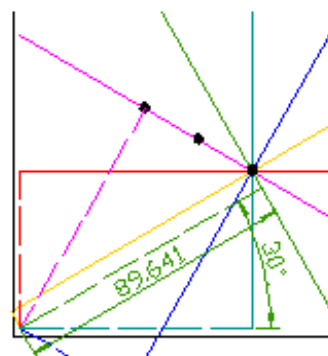
- Transformación *espacio de entrada* → *espacio paramétrico*:



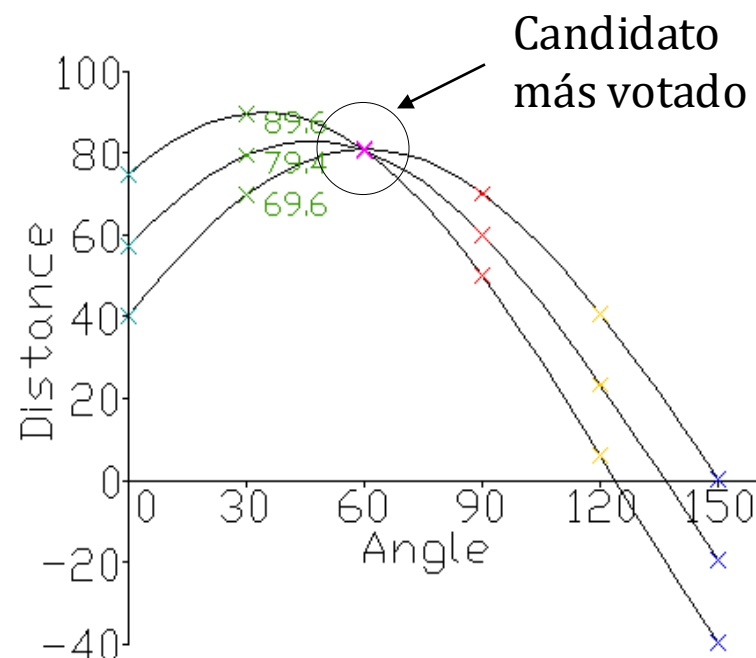
Angle	Dist.
0	40
30	69.6
60	81.2
90	70
120	40.6
150	0.4



Angle	Dist.
0	57.1
30	79.5
60	80.5
90	60
120	23.4
150	-19.5



Angle	Dist.
0	74.6
30	89.6
60	80.6
90	50
120	6.0
150	-39.6



Demo interactiva: [aquí](#)

Apéndice T4: Estimación robusta

Algoritmo básico HT (para rectas)



Paso 1; Inicializar $H[\theta, r]=0$.

Paso 2: Iterar sobre la imagen:

Para cada punto (x, y) de la imagen

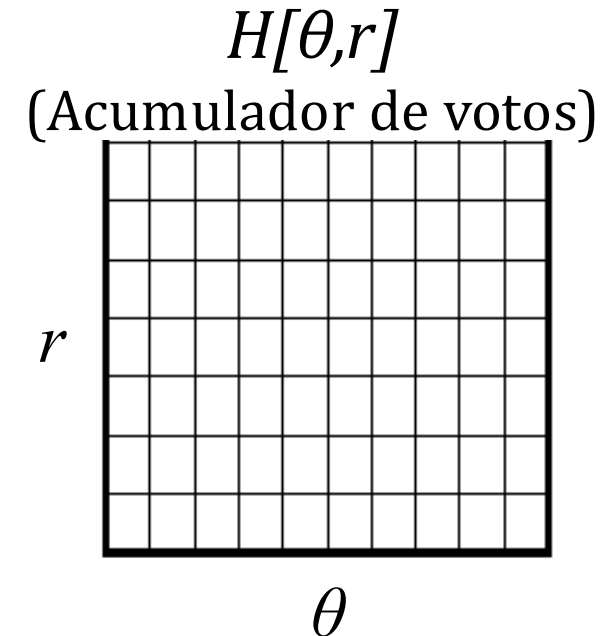
Para cada θ (con la cuantización deseada)

$H[\theta, r] += 1$, con

Paso 3: Encontrar máximo local de $H[\theta, r]$.

Paso 4: La línea (θ_{max}, r_{max}) se marca como encontrada, y (opcionalmente) se eliminan los votos asociados.

Paso 5: Repetir pasos 2-4 hasta que el máximo de votos de la celda $(\theta_{max}, r_{max}) < umbral$.

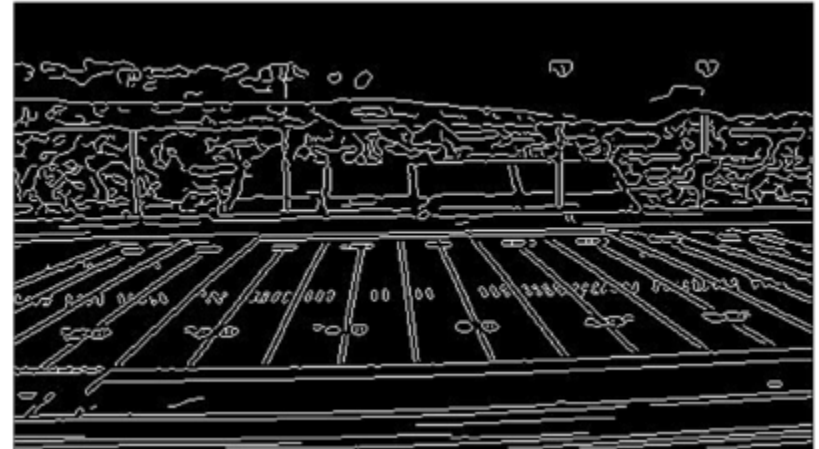


Apéndice T4: Estimación robusta

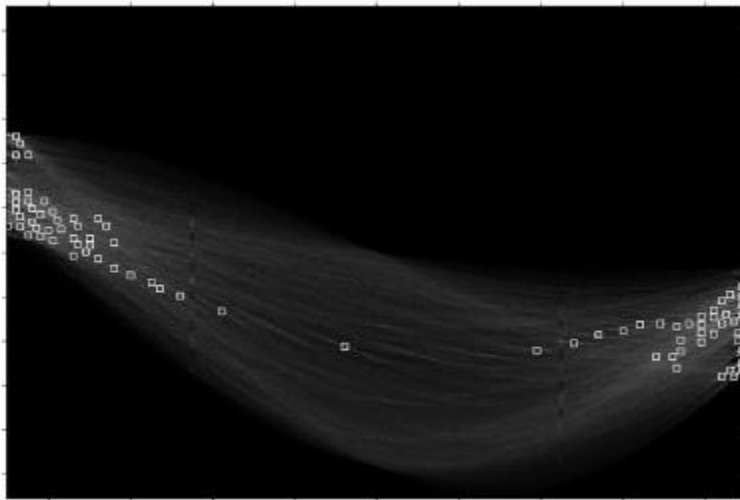
Ejemplo de resultados HT en detección de segmentos



Original



Puntos de borde (p.e. Canny)



Espacio paramétrico
de votación



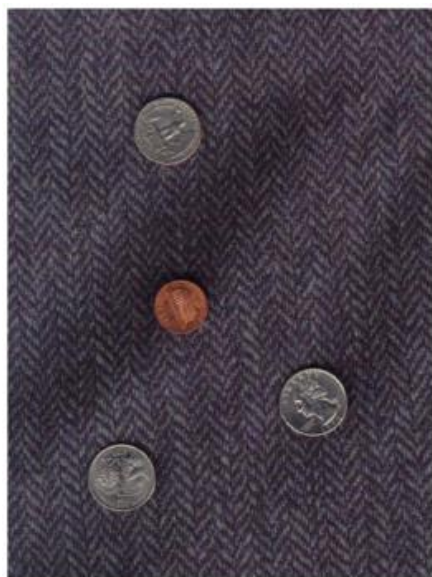
Segmentos (tras acotar proyección de
agrupaciones de puntos sobre rectas
encontradas)

Apéndice T4: Estimación robusta

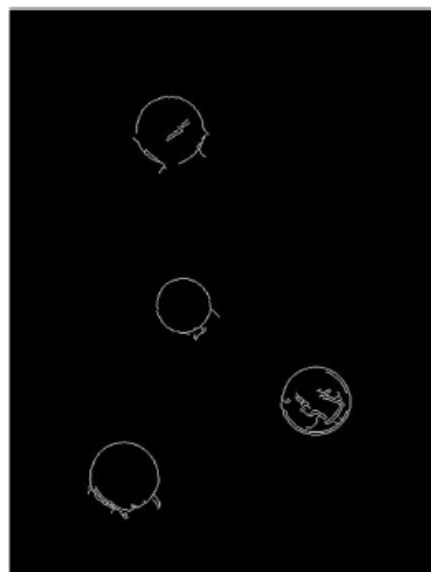
Generalización de la HT



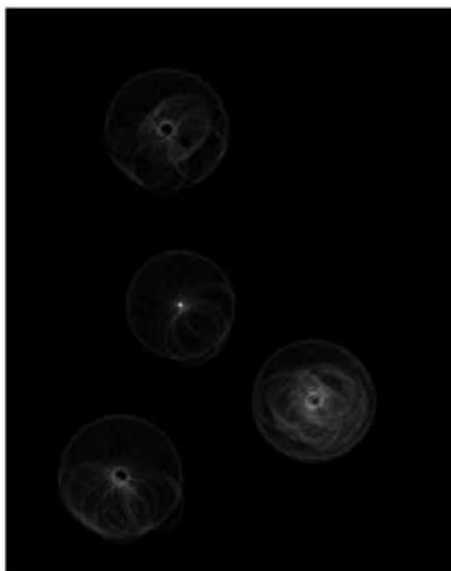
- ▶ La HT es fácilmente trasladable a otro tipo de estructuras (círculos, elipses, ...), cambiando el espacio de parámetros.
- ▶ Ojo con el posible crecimiento del espacio paramétrico.
- ▶ Ejemplo detección de círculos (de radio conocido; si no lo es el espacio de parámetros es 3D → más costoso):



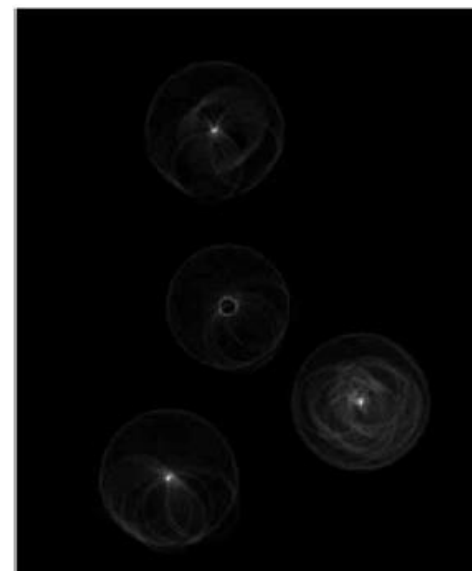
Original



Bordes



HT (radio penique)



HT (radio cuarto
de dólar)



► A tener en cuenta:

- **Eficiencia:** un punto de entrada → ¿múltiples votos?
→ intentar sólo uno (ahora veremos posibilidades).
- **Discretización** del espacio de parámetros (= espacio de votación):
 - Demasiado “gruesa” → poca precisión en modelos encontrados.
 - Demasiado “fina” → poca agrupación en los votos.
- A veces, mejor tratar votos simplemente como listas de puntos → necesario ***clustering*** robusto en el espacio de parámetros.

- ▶ HT con gradiente: un punto + una orientación → un sólo voto.
 - Una variante es la HT conectiva, que también pretende un sólo voto, pero no usando el gradiente, sino agrupaciones locales de píxeles alineados.
- ▶ HT probabilística: trabajar sólo sobre un subconjunto aleatorio de los puntos de entrada
 - La solución no varía mucho, y se gana en eficiencia.
- ▶ HT aleatoria: Ir cogiendo los puntos de dos en dos, aleatoriamente → Cada par de puntos es un sólo voto.
 - Al iterar este proceso, llega un momento en que una celda excede el umbral → se detecta una recta, se borran los puntos correspondientes y se continúa.

Apéndice T4: Estimación robusta

HT vs RANSAC: Ventajas e inconvenientes



- ▶ Ambos son algoritmos genéricos de **estimación robusta** de modelos en presencia de *outliers* en los datos.
- ▶ **HT** tiene los problemas de elección de parámetros, discretización del espacio y crecimiento de éste para modelos con muchas dimensiones...
- ▶ ... pero puede soportar % de *outliers* muy elevados.
- ▶ **RANSAC** no necesita discretización del espacio de parámetros...
- ▶ ... pero a mayor % de *outliers* se puede disparar su costo.

- ▶ Entrada Wikipedia: “*Hough transform*”:
 - En la [página](#) hay una explicación detallada de su filosofía y funcionamiento, así como algunos enlaces interesantes para mostrar su funcionamiento y sus posibles extensiones.

- ▶ Entrada Wikipedia: “*Random Sample Consensus*”:
 - En la [página](#) hay una implementación en pseudocódigo, incluyendo la explicación del estudio estadístico para razonar sobre el mínimo número de muestreos necesarios.